

# Day 12 Learning (25/03/2026)

## 1. SPRING FRAMEWORK OVERVIEW

The Spring Framework is an open-source Java platform that provides comprehensive infrastructure support for developing Java applications. It handles the "plumbing" of the application so developers can focus on the business logic.

### Core Concepts

- **Inversion of Control (IoC):** Instead of the developer manually creating objects (using the 'new' keyword), the control is handed over to the Spring Container. The container manages the lifecycle of objects.
  - **Dependency Injection (DI):** This is the pattern used to implement IoC. Spring "injects" the required dependencies (objects) into a class at runtime.
  - **Aspect-Oriented Programming (AOP):** Used to separate secondary tasks like logging, security, and transaction management from the main business logic.
- 

## 2. SPRING BEANS AND CONTAINER

- **Spring Bean:** Any Java object that is initialized and managed by the Spring IoC container is called a Bean.
  - **IoC Container:** The "environment" where beans live. It is responsible for instantiating, configuring, and assembling the beans.
  - **ApplicationContext:** This is the advanced Spring Container. It loads the bean definitions and wires them together. It is the "Brain" of the Spring application.
- 

## 3. SPRING VS. SPRING BOOT

- **Spring Framework:** Requires manual configuration (XML or Java Config), requires an external server (like Tomcat) to be set up, and involves a lot of boilerplate code.
- **Spring Boot:** An extension of Spring that offers **Auto-Configuration**. It comes with an **Embedded Server** (Tomcat is built-in) and uses **Starters** to reduce build configuration. It is designed for "Production-ready" applications.

---

## 4. MAJOR SPRING ANNOTATIONS

### Bean Management

- **@Component:** Marks a class as a Spring Bean. Spring will scan and manage this class.
- **@Service:** A specialized form of **@Component** used for business logic classes.
- **@Repository:** Used for the Data Access Layer (DAO). It handles database exceptions.
- **@Autowired:** Used to inject a dependency automatically. Spring finds the correct bean from the container and provides it to the class.
- **@Bean:** Used on methods inside a Configuration class to define a bean manually (useful for third-party libraries).

### Web & REST API

- **@RestController:** A combination of **@Controller** and **@ResponseBody**. It tells Spring that the class will handle HTTP requests and return data (JSON) directly.
- **@RequestMapping:** Maps web requests to specific controller methods. (Shortcuts: **@GetMapping**, **@PostMapping**).
- **@RequestBody:** Converts the incoming JSON request body into a Java Object.
- **@PathVariable:** Used to extract values directly from the URL (e.g., `/users/{id}`).

---

## 5. REQUEST FLOW AND JSON CONVERSION

### The Execution Flow

1. **Request Arrival:** The HTTP request (like a POST) hits the **DispatcherServlet**.
2. **Mapping:** The **DispatcherServlet** consults the **HandlerMapping** to find the right **Controller**.
3. **Conversion:** If the request contains JSON, the **Jackson Library** (built into Spring Boot) performs **Deserialization**—it converts the JSON string into a Java Object using the **@RequestBody** annotation.
4. **Service/Data:** The Controller calls the **Service** bean, which might call the **Repository** bean to talk to the database.

5. **Response:** The method returns a Java Object. Jackson performs **Serialization**—converting the Java Object back into JSON to send to the user.
- 

## 6. BEAN SCOPES

- **Singleton (Default):** Only one instance of the bean is created per Spring Container.
  - **Prototype:** A new instance is created every time the bean is requested.
  - **Request:** One instance per HTTP request (Web apps only).
  - **Session:** One instance per HTTP session (Web apps only).
- 

## 7. EXCEPTION HANDLING

- **@ControllerAdvice:** A global interceptor for exceptions. It allows you to handle errors from all controllers in one central place, ensuring a consistent error response format (JSON) for the user.
- 

## #SPRING BEAN SCOPES: SINGLETON & PROTOTYPE

### 1. IS EVERY SPRING BEAN A SINGLETON?

By default, **YES**. In Spring, every bean you define (using `@Component`, `@Service`, etc.) has a **Singleton scope** unless you explicitly change it.

- **Definition:** Only one instance of the bean is created per Spring IoC Container.
  - **Behavior:** When multiple components request or `@Autowired` the same bean, Spring provides the exact same object reference to everyone.
  - **Purpose:** To conserve memory and improve performance by reusing stateless objects (like Services and Repositories).
- 

### 2. SPRING SINGLETON VS. JAVA SINGLETON

It is important to understand that a "Spring Singleton" is different from the "Gang of Four (GoF) Singleton Design Pattern."

- **Java Singleton:** The class itself controls the instantiation (usually via a private constructor). It is one instance per **ClassLoader**.
  - **Spring Singleton:** The container controls the instantiation. The class is a normal Java class, but the **Spring Container** ensures only one object is created within its own context.
- 

### 3. PROTOTYPE SCOPE

If you need a new object every time you use it, you use the **Prototype** scope.

- **Annotation:** `@Scope("prototype")`
  - **Behavior:** Spring creates a brand new instance of the bean every time it is requested from the container.
  - **Responsibility:** Unlike Singleton beans, Spring does not manage the complete lifecycle of a Prototype bean (it doesn't call the "Destroy" method).
- 

### 4. TRICKY INTERVIEW QUESTIONS (FAQ)

#### Q1: Are Spring Singleton beans Thread-Safe?

**Answer:** No. Spring does not provide thread safety for singleton beans. Since the same object is shared across all threads (requests), if you use "instance variables" (fields) to store user-specific data, you will face data inconsistency.

- **Best Practice:** Singleton beans should be **Stateless** (they should not store data in variables).

#### Q2: What happens if a Singleton bean has a Prototype bean injected into it?

**Answer:** This is a very common tricky question! Since the Singleton bean is only created **once**, its dependencies are also injected only **once**.

- Even if the injected bean is marked as `@Scope("prototype")`, it will stay the same instance throughout the life of the Singleton bean.
- **Solution:** Use **Lookup Method Injection** or the `ObjectFactory` interface if you need a fresh Prototype bean inside a Singleton.

#### Q3: Does Spring's Singleton scope mean there is only one object in the entire JVM?

**Answer:** No. It is one object **per Spring ApplicationContext (Container)**. If you run two different Spring Containers in the same JVM, they will each have their own instance of the bean.

#### Q4: Why is Singleton the default scope in Spring?

**Answer:** Most classes in an enterprise application (like Controllers, Services, and DAOs) are **Stateless**. Reusing a single instance reduces the overhead of frequent object creation and Garbage Collection (GC) pressure.

---

#### 5. COMPARISON TABLE

| Feature         | Singleton Scope          | Prototype Scope                      |
|-----------------|--------------------------|--------------------------------------|
| Object Creation | Once per Container.      | Every time it's requested.           |
| Default?        | Yes.                     | No.                                  |
| Memory          | Efficient (low memory).  | Higher memory usage.                 |
| Lifecycle       | Fully managed by Spring. | Spring doesn't call <b>Destroy</b> . |
| Usage           | For stateless services.  | For objects with "state" (data).     |